

Genestrip: Space-efficient k -mer counting, read classification and filtering for selected groups of organisms

Daniel Pfeifer^{1*} and Clas Rurik²

^{1*}IT Faculty, Heilbronn University, Max-Planck-Str. 39, Heilbronn, 74081, Baden-Württemberg, Germany.

²codeclap, Kokhaven 20, Røddekro, 6230 Denmark.

*Corresponding author(s). E-mail(s): daniel.pfeifer@hs-heilbronn.de;
Contributing authors: cru@codeclap.cc;

Abstract

Background: The consumption of main memory resources is a significant burden in k -mer-based metagenomic analysis when creating related databases but also when performing (unique) k -mer-counting and read classification. Genestrip addresses this issue by focusing on small but freely configurable groups of organisms. Regarding the selected organisms, Genestrip produces k -mer databases and results comparable to those of KrakenUniq but at a fraction of its required memory resources. Our tool ensures that during database generation, the most suitable lowest common ancestor taxon is assigned for each stored k -mer by *also considering genomes of organisms whose k -mers are not included in the database*. This enables read analysis with high accuracy for the organisms of interest. Moreover, Genestrip can also be employed to efficiently and accurately filter fastq files with regard to reads from the chosen organisms.

Results: We assessed the correctness, usefulness and performance of Genestrip in different contexts and showed that it indeed ascertains high accuracy for organisms whose genomes were included in a corresponding database. Our example databases comprise millions to a few billions of k -mers covering a dozen to a few thousands of species and lend themselves to usage in tick surveillance, medical diagnostics or agriculture. All databases were generated on a regular PC within hours, and related analysis performance was competitive to highly favorable. The deliberate focus on a particular set of genera or species allowed for more genomes to be included from related organisms while the resulting databases remained small. We exemplify, that such small but deep databases tend to improve recall during read classification while sustaining high precision.

Conclusions: Due to Genestrip’s particular way of updating the k -mers’ lowest common ancestor taxa, both database creation and fastq file analysis can be realized with little memory and with favorable runtimes as well as high analysis accuracy. So both, database creation *and* read classification may be performed even on regular PCs. Genestrip’s qualities empower users to flexibly design, build and use small k -mer databases for their own needs with potentially deep genomic coverage.

Keywords: Metagenomics, k -mer counting, read classification, small k -mer database, fastq file filtering, high accuracy, high performance

1 Background

1.1 Introduction

With the availability of low-cost and small-scale sequencing devices, metagenomic analysis will likely replace many established technologies for the detection and surveillance of specific groups of microorganisms. Related applications may affect various areas such as farming, ecology, food processing and medicine. While sequencing devices have become ready for ubiquitous use, software tools for metagenomics still tend to be resource intensive and too hard to use for end users.

To analyze large sets of reads, software tools based on k -mer matching and counting such as Kraken 1 [1], Kraken 2 [2] or KrakenUniq [3] are at hand. With regard to pathogen detection, KrakenUniq is particularly suited because of its very low false positive rate but also because of its efficiency and high sensitivity [3].

1.2 Problem statement

In order to avoid false positive classifications of reads, k -mer counting tools resort to very large databases, containing millions of encoded k -mers along with their respective taxa. A database is usually loaded entirely into main memory and often consumes tens of gigabytes of space. This requirement mandates specialized and expensive compute servers with very large main memory.

In recognition of this problem, KrakenUniq has been extended for database processing in chunks at read analysis time [4]. However, database creation still mandates ultralarge main memory: For example, in 2018 the production of a database for Kraken 1 and, equivalently, KrakenUniq that just covers the bacterial genomes from the RefSeq [5] “required ca. 2.5 TB of RAM and ca. 11 days” on a high-performance computer [6]. However, since then, the RefSeq’s number of prokaryotic genomes alone has increased by a factor of 4.5 to over 38 Mio. [7]. Therefore, related organisms are often not covered below the species rank and also genomes below the quality level “complete genome” are usually not included in a corresponding database.

Despite the fact that these databases may not cover organisms of interest at the necessary genomic depth, users tend to apply them because otherwise, creating even larger databases would require even more main memory [6]. So, although a much richer

set of genomes for organisms of interest may be available in principle – for example via recent versions of the RefSeq or in addition via Genbank [8] – these genomes cannot easily be leveraged for metagenomic analysis [9, 10].

Even though Kraken 2 offers more memory efficient k -mer databases, *it is not a suitable alternative in fields such as medical diagnostics where precision of read classification is paramount* [4]. This is because in comparison with Kraken 1 and KrakenUniq, Kraken 2 produces smaller databases but it compresses stored k -mers under loss of information, therefore facilitating false positive k -mer counts [2, 4].

1.3 Approach

If only a smaller group of a few dozen genera or a few 100 species is under consideration, then it seems promising to simply *generate a smaller database that contains just the k -mers from respective genomes*. However, *without additional measures, this very likely leads to many false positives*: The reason for this is that owing to the evolutionary relationship of organisms, *a large fraction of k -mers that belong to the genome of a species are unspecific* in the sense that they can as well be found in the genomes other species or genera not considered for analysis. Thus, when using too small a database, a read may be falsely assigned to a species because it could consist of k -mers that *also* belong to another species’ genome not included in the database.

In order to enable a smaller database with no false positives emerging from this problem, we introduce Genestrip, as new metagenomic software tool supporting (unique) k -mer counting, read classification and read filtering.

In principle, Genestrip employs a *two-phase* database generation process: During the *entry phase*, all k -mers of genomes from selected organisms are entered into the database on the basis of the RefSeq [5] and optionally GenBank [8] and additional sources. During the *update phase*, all stored k -mers from the database are checked against the genomes of *all* other organisms from the RefSeq belonging to the same *category*¹. So, if a stored k -mer also occurs in another organism which is not part of the database, the k -mer’s taxon still gets updated in the database and the lowest common ancestor’s taxon (known as LCA) is entered instead [6]. This prevents a k -mer from being falsely associated with one of the selected organisms given that it is too unspecific.

Based on this approach, Genestrip achieves a low memory footprint during database creation as well as at read analysis time. Moreover, it offers competitive to highly favorable runtimes for k -mer matching and read classification. Together, these qualities empower users to create and use respective databases on machines with regular main memory size. Genestrip’s k -mer counting results are similar to those of KrakenUniq [3] with respect to the selected organisms. Genestrip also supports unique k -mer counting with exact counts and with low memory overhead.

In the results section of this paper, we demonstrate the usefulness of our approach by introducing a small Genestrip database for tick-borne bacterial pathogens. An envisioned application is tick surveillance via metagenomics as reported in [11]. We performed analyses of tick-related fastq files on a regular PC in less than a minute.

¹By standard, the RefSeq is structurally divided into 14 categories including “Bacteria”, “Fungi”, “Invertebrate”, “Protozoa”, “Viral” and others.

Thanks to the small database’s high genomic coverage for the organisms of interest, we achieved considerably better recall than with a large standard database under KrakenUniq. This means we found more k -mers and matching reads for such organisms while also sustaining high precision. As part of our performance experiments, we briefly present further small databases that may prove useful in agriculture and infectious disease diagnostics.

Genestrip is written in Java, has a small code base and supports command-line-based as well as API-based usage. Based on the Java Virtual Machine, it can be executed on modern operating systems including Linux, Windows and macOS.

2 Implementation

2.1 System structure

Genestrip’s database generation mandates the setting of groups of organisms whose genomes will be exclusively represented in a resulting database. Users choose the desired genera, species or strains via a configuration file by listing the corresponding taxa. If higher ranked taxa are entered, then by default, *all* of the descending taxa will be included and k -mers of their genomes will be added to the database accordingly.

Genestrip generates a database for a selected group of organisms primarily via genomes from the RefSeq [5] but genomes from Genbank and additional genomic files may be included on top or instead. To better handle the complexity of the generation process, it employs a goal-oriented approach similar to Make [12]: Upon access, each goal returns a file or an object in main memory. A goal is executed only if its result is not yet available. Goal execution may involve the access of other goals which in turn, may trigger their execution. Respective goal dependencies are explicitly specified in terms of a directed acyclic graph.

Figure 1 depicts an extract of Genestrip’s goal dependency graph: Several goals (prefixed with **d**;) revolve around automatically downloading all necessary files of the RefSeq, Genbank and other data from the NCBI’s Web site [13]. When required for database generation, the goal **refseqfna** downloads the RefSeq genomes belonging to a matching category as a collection of zip files. The collection is one of NCBI’s standard offerings. By consulting the goal **checksummap**, **refseqfna** also performs an MD5 fingerprint check after each zip file download to ensure data integrity and reattempts the download in case the initial file appears to be corrupted.

Several object goals (prefixed with **o**;) load files and create data structures or compute results in main memory for further processing. Most file goals (prefixed with **f**;) write computational results to files. Boxes with edged corners depict goals that are meant to be executed directly via the command line. For example, the goal **match** is for fastq file analysis but it would trigger a respective database generation goal if the corresponding database file was missing.

Solid edges symbolize “eager” dependencies where the existence of a dependency’s result is ensured prior to the execution of the dependent goal. In contrast, dotted edges symbolize “lazy” dependencies where the existence of a dependency’s result is ensured when first accessed during the execution of the dependent goal.

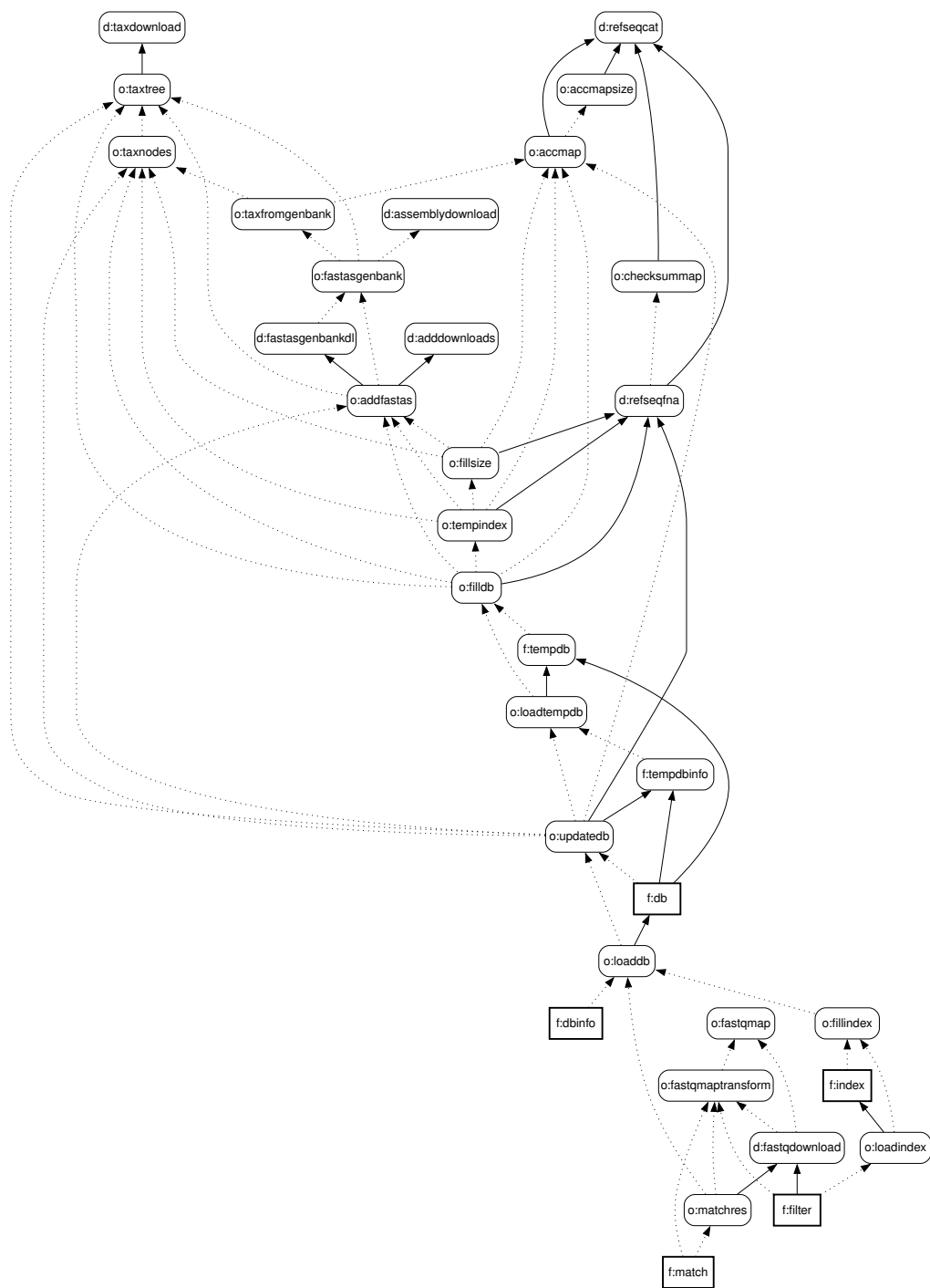


Fig. 1 Extract of Genestrip's goal dependency graph

2.2 Database generation

Genestrip stores a database primarily as a large array L of sorted long values. Each long value encodes a k -mer from the genome of a selected organism. The default for k is 31 but smaller values and $k = 32$ are possible. In order to handle reverse complements of reads later at read analysis time, stored k -mers are standardized in the following way: Each k -mer is encoded twice as a long value — once in its regular version and once via its reverse complement — but only the larger of the two encoded values is stored in the database.

A second array of the same size as L encodes taxa for each k -mer in L . At read analysis time, lookups in L are performed via a binary search. However, an additional Bloom filter [14] also contains all stored k -mers. By default, it is consulted upfront to check whether a certain k -mer might be in L . A negative result avoids a more expensive binary search in L and therefore saves overall execution cost. Section 2.3 explains this type of filter in more detail.

Moreover, all parts of the NCBI taxonomy tree [15] required at analysis time are also included in the database. The taxonomy extract comprises entries for the selected organisms and all of their ancestors. At analysis time, a bit vector with the same size as L indicates which k -mers from L were found at least once. This is sufficient for exact unique k -mer counting and is space-efficient, as it requires only about 1.6% of additional main memory with respect to L .

Table 1 describes the most relevant goals for database generation in their typical execution order. All of these goals are multithreading enabled for faster database generation.

Additional genomes can be manually added either to create deeper databases or, else, to filter out contaminating sequences that are mistakenly included in the genome of an organism of interest. For example, [16] reported that human DNA fragments often contaminate bacterial genomes for various reasons whereas [17] discussed a pipeline to remove contaminants from genomes of eukaryotic pathogens.

Regarding database generation, k -mers from contaminating sequences can be identified during the update phase: For example, the human genome can be included via configuration in the update phase only; thus, a k -mer from the human genome that is also found in the contaminated genome will be pushed to the lowest common ancestor taxon of both species. Still, as the human genome is employed during the update phase only, a resulting database does not include k -mers from the human genome (other than those from contaminating sequences). This ensures a small-sized database.

Of note, Genestrip allows for the exclusion of k -mers with low complexity known as genomic dust [18]: As part of the entry phase, each k -mers's repetitiveness in terms of base pairs is measured via a fibonacci-based function. If the corresponding value exceeds a configurable threshold then the k -mer is dropped.

2.3 Bloom filter generation

Besides k -mer matching and read classification, Genestrip supports filtering of fastq files via a Bloom filter in similarity to [19]. When generated via the goal `index`, the Bloom filter gets stored in a separate file. The goal `filter` loads the filter and

Goal name	Description
fillsize	Determines the maximum size of the database in terms of number of k -mers by reading the respective reference genome files and including the genomes of selected organisms. This results in a conservative estimate where duplicates are double counted.
tempindex	Creates a Bloom filter filled with all k -mers of the genomes of selected organisms. The Bloom filter's size is initially set via the result from fillsize . As the Bloom filter avoids double counting, its final number of entries serves as a more exact basis for the actual database size.
filldb	This <i>entry phase</i> populates the database with all k -mers of genomes from selected organisms. The appropriate size of the database is a close but conservative estimate based on tempindex . Duplicate k -mer entries are avoided via a Bloom filter that tracks the already stored k -mers. Finally, the stored k -mers are sorted in order to enable binary search at analysis time but also for the goal updatedb from below.
updatedb	This <i>update phase</i> updates the populated database to account for common k -mers between organisms. All genomes from the RefSeq belonging to a set of preconfigurable taxonomic categories are considered during this phase. This ensures that a k -mer from a selected organism obtains the correct taxon as the k -mer may also occur in the genome of another organism (which might be represented in the database or not). In the finished database, the respective k -mer is associated with the lowest common ancestor taxon as computed via all considered genomes that contain the k -mer.

Table 1 Genestrip's most relevant goals for database generation

processes fastq files accordingly. The standard configuration of the filtering process rewrites a read to the output if the read contains at least one k -mer that is recognized by the filter since because of this, the read may belong to a selected organism of interest.

A Bloom filter is a probabilistic data structure to efficiently test whether or not a data object got stored [14]. It generally depends on n hash functions $h_1, \dots, h_n$ that ideally should be statistically independent from each other. In our case, the input to each hash function is a k -mer encoded as a long value l and its output is an index of a bit in a large bit vector b which forms the Bloom filter's store. When the value l gets stored, the bits $b[h_1(l)], \dots, b[h_n(l)]$ are set but any other bits in b remain untouched. To test whether l is contained in the filter, one simply checks whether $b[h_1(l)] \wedge \dots \wedge b[h_n(l)]$ is true. False positives from this check may occur since the storing of values other than l may have caused the corresponding bits to be set. If the number of data objects to be stored is known, then the false positive rate can be optimized via n and the size of b , but this also demands the independence of the h_i [14].

A typical challenge is to determine a method that generates an arbitrary number of h_i which is needed since n may vary: On the one hand, the computation of $h_i(l)$ should be very efficient but also, the Bloom filter's false positive rate must not suffer too much, if true independence cannot be guaranteed for the h_i . Regarding the storing of k -mers we found by trial and error that the simple generator method

$$h_i(l) = ((l \oplus s_i) \bmod |b|) + 1$$

works well in practice. Hereby, $\oplus$ is the bitwise xor-operator, $s_1, \dots, s_n$ are randomly initialized long-valued salts and $|b|$ is the size of b . Functional testing showed that this method indeed keeps the false positive rate close to its theoretical expectation.

Genestrip benefits from the efficiency of the resulting Bloom filters when filtering fastq files but also during database generation and when classifying reads. It populates such filters as part of the goals `tempindex`, `filldb` and `index` from Table 1 and Figure 1, respectively.

2.4 Read classification

The goal `match` loads a database and performs the read classification on fastq files. The principal classification algorithm is similar to that from Kraken 1 [1] and equivalently KrakenUniq. When iterating over a read’s k -mers, each k -mer is encoded as a long – once by considering its regular order and once via its reverse complement order. For efficiency, only the larger of the two encoded values is looked up in the database. This is sufficient since, if at all, then only just the larger of two values got stored in the first place during database generation (see Section 2.2).

Genestrip implements a multithreading enabled classification procedure following a producer consumer pattern, where a producer thread reads a fastq file and puts reads into an in-memory queue while consumer threads pick reads from the queue in order to classify them.

Two types of error thresholds can be configured for read classification:

1. The ratio or number of k -mers per read that must at least be found in the database and
2. the ratio or number of k -mers per read that must be consistent with the read’s designated class taxon. A k -mer is consistent with a read’s class taxon if the k -mer’s taxon as stored in the database is the same or if it is a taxonomic ancestor of the class taxon.

The thresholds allow for playing off recall against precision in light of potentially high error rates of some sequencing devices [20, 21]. For all the experiments throughout this paper, both thresholds stick to the default of one k -mer which is the most sensitive level.

2.5 Analysis reports

The read classification via the goal `match` produces analysis reports in CSV file format with a comprehensive set of statistical values. We use the following notation to describe them: Let $k_{t,f,d}$ be the number of k -mers for taxon t found in fastq file f with respect to database d and let $u_{t,f,d}$ be the corresponding number of unique k -mers. Similarly, let $u_{t,d}$ be the number of (unique) k -mers for t stored in d .

Table 2 describes the most important reported columns. Additional columns exist to accumulate k -mer counts and read counts etc. upward to ancestors in the taxonomy. We also track the standard deviations of `avg. contig length`, `avg. read length`, `mean class error` and other columns wherever suitable. Moreover, normalized values derived from the original values by dividing by $u_{t,d}$ are included where suitable.

Normalized values aim to reduce counting bias due to a high or low representation of a taxon t in terms of its stored k -mers $u_{t,d}$.

Column name	Description
<code>taxid</code>	The class taxon t
<code>reads</code>	The number of reads classified with respect to t
<code>kmers from reads</code>	The number of k -mers from reads for t that are consistent with t
<code>kmers</code>	$k_{t,f,d}$
<code>unique kmers</code>	$u_{t,d}$
<code>avg. contig length</code>	The average length of contiguous sequences of k -mers that are specific to t 's genome(s) as represented in d
<code>max contig length</code>	The maximum length of all contiguous sequences of k -mers that are specific to t 's genome(s)
<code>avg. read length</code>	The average length of reads for t in base pairs
<code>db coverage</code>	$u_{t,f}/u_{t,d}$
<code>mean class error</code>	The mean ratio of k -mers from a read r for t , where those k -mers are not consistent with t per r 's total number of k -mers

Table 2 Extract of columns from Genestrip's read analysis reports

3 Results

3.1 Correctness

To assess Genestrip's correctness with regard to k -mer counting and read classification, we compared example databases and respective analysis results directly and in detail with those from KrakenUniq. We consulted KrakenUniq because Genestrip aims to provide a similar k -mer based functionality while also enabling small databases without loss of precision and recall. Moreover, KrakenUniq and related tools are trusted by a large user base in bioinformatics [22].

As part of our test procedure, we created database contents under both systems that were as identical as possible in terms of their genomic basis and their stored k -mers. We then applied the created databases to suitable fastq files in order to compare the results between both systems. Highly similar results from this check mutually solidify the correctness of both systems since they had been developed completely independently. (In addition, Genestrip's correctness is also supported via extensive automated functional tests with high test coverage as part of its code base.)

To create the databases, we chose all RefSeq genomes from the category "Viral". Figure 2 compares the structure of the two databases produced with KrakenUniq (with 373,250,262 k -mers in total) and Genestrip (with 374,412,041 k -mers in total). The databases are nearly identical with regard to the number of k -mers stored per taxon with 15 genomes and respective taxa occurring in KrakenUniq but not in Genestrip and 33 genomes vice versa.² The slightly different sets of genomes resulted in 158

²The differences between the sets of selected genomes are caused by different approaches on how the genomes are preselected and downloaded by KrakenUniq and Genestrip, respectively. They do not constitute erroneous behavior of either system.

differences in k -mers stored per taxon. A large part of the differences is a consequence of the necessary lowest common ancestor update on stored k -mers (see Section 1). In the following, we name the two databases `cv_ku` and `cv_g` with regard to KrakenUniq and Genestrip, respectively.

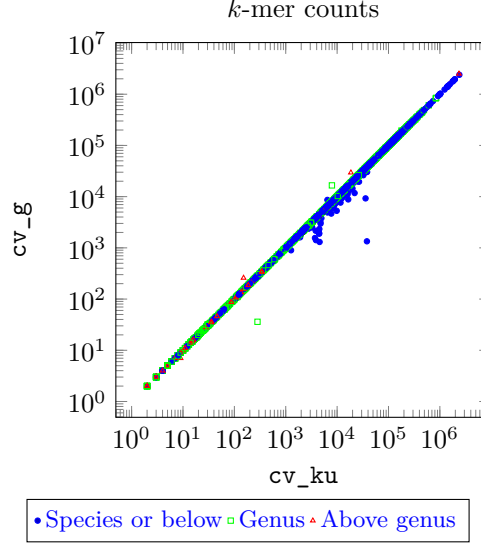


Fig. 2 Scatter plot of k -mer counts for the RefSeq databases `cv_ku` from KrakenUniq and `cv_g` from Genestrip covering the entire category “Viral” (based on a logarithmic scale with 1 added to each value in order to enable the representation of zero-counts). The 16,087 depicted entries cover all of the taxa present in both databases.

We applied the two databases to a simulated fastq file extracted from [23] where the file’s reads were used to benchmark metagenomic pipelines aimed at the discovery of new viruses. As the ground truth is known, this allowed for a similarity comparison but also for checking classification accuracy. The file contains 61,550 long reads extracted from 6,155 viral standard genomes from the RefSeq [23].

Figure 3 shows scatter plots for k -mer counts and read counts from both tools on a logarithmic scale. Each data point represents a nonzero count from KrakenUniq or Genestrip with respect to a taxon. As shown in (a), the k -mer counts only differ in 120 out of a total of 6,245 data points. The differences are rooted in the slightly different database contents as mentioned above. The classified read counts in (b) are nearly identical with just 14 differences. As stated before, this high agreement mutually solidifies the correctness of both systems, but obviously, it does not yet highlight the advantages of Genestrip when creating and using small databases for selected groups of organisms. Both tools scored a read classification accuracy of 99.4% for the genus rank and 95.2% for the species rank.

As real-world examples, we applied the two databases to short-read fastq files from DNA-sequencing human saliva of five different donors. Table 3 shows statistics of the fastq files and states the number of data points, i.e., the taxa for which at least one

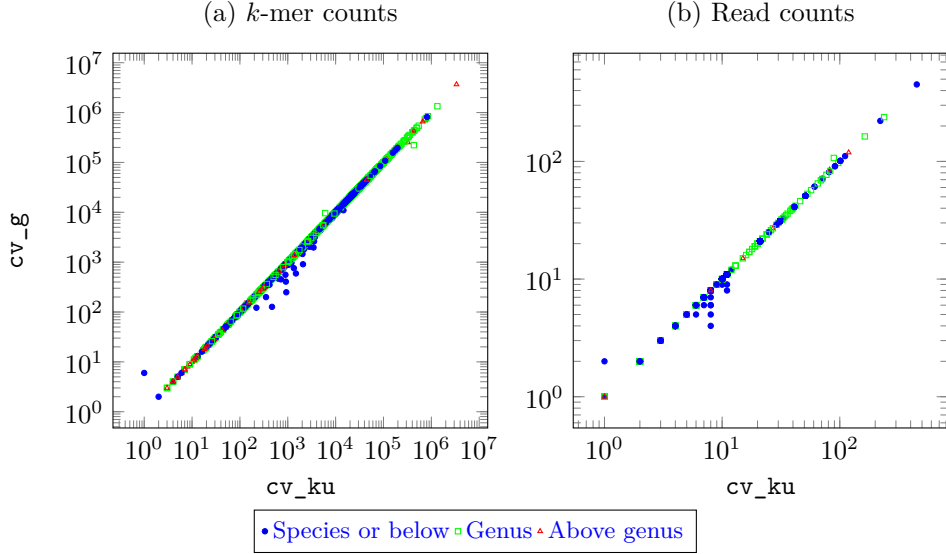


Fig. 3 Scatter plots of k -mer counts (a) and read counts (b) for a simulated viral fastq file from [23] analyzed with KrakenUniq and Genestrip using the viral databases `cv_ku` and `cv_g` from Figure 2 (based on a logarithmic scale with 1 added to each value in order to enable the representation of zero-counts).

k -mer was counted via either KrakenUniq or Genestrip. The last column indicates where results differed by at least one between KrakenUniq and Genestrip. As can be seen, the counts of the very most k -mers are identical. For visualization, Figure 4 shows scatter plots of k -mer counts and read counts from KrakenUniq and Genestrip on a logarithmic scale for one of the fast files.

#	Reads	Avg. BPs per read	Data points	Differing k -mer counts (%)
1	824,479,570	429	1,894	11 (0.58)
2	605,561,636	685	1,346	10 (0.74)
3	900,709,176	191	1,758	13 (0.74)
4	980,879,835	276	1,790	8 (0.45)
5	812,085,208	101	1,767	16 (0.91)

Table 3 Number of differences per number of data points for k -mer counts on the basis of real-world fastq files from DNA-sequencing five human saliva samples analyzed via the KrakenUniq viral database `cv_ku` and the Genestrip viral database `cv_g`, as shown in Figure 2. A data point corresponds to a taxon for which at least one k -mer was counted via either KrakenUniq or Genestrip.

3.2 Accuracy in the context of small databases

The following results highlight the importance of the update phase according to Section 1 in the context of small databases.

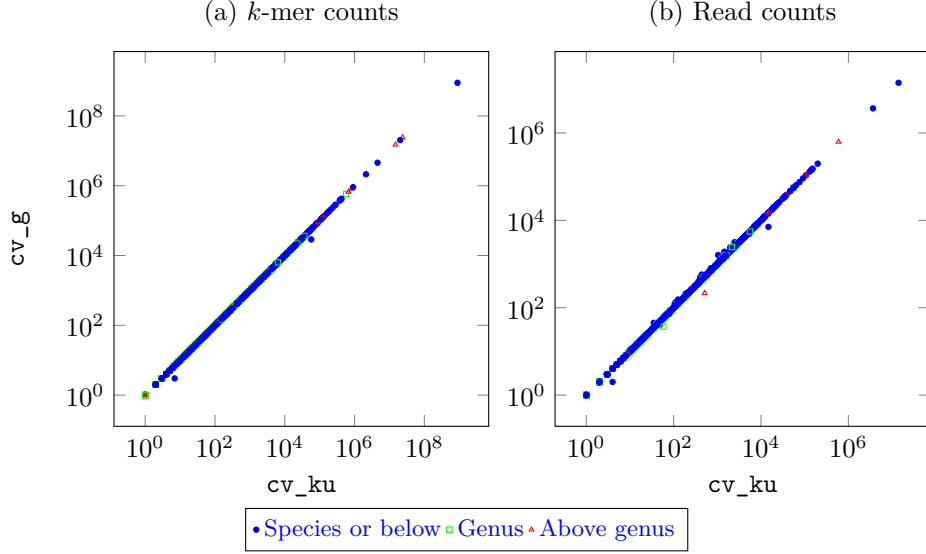


Fig. 4 Scatter plots of k -mer counts (a) and read counts (b) for fastq file #3 from Table 3 using the viral databases from Figure 2 (based on a logarithmic scale with 1 added to each value to enable the representation of zero-counts).

As a first example, we created two small viral databases based on the RefSeq covering *human viruses only*³, again under both KrakenUniq and Genestrip. In the following, we name the two databases *hv_ku* and *hv_g* with regard to KrakenUniq and Genestrip, respectively. We again assured that the databases were as identical as possible in terms of their genomic basis and their stored k -mers. Figure 5 compares the viral database contents for the databases from Section 3.1 with the respective “human virus only” database for (a) KrakenUniq and (b) Genestrip. The plots depict data points of the 407 taxa referenced in both databases.

Regarding Genestrip’s databases from Figure 5 (b), all k -mer counts on the species rank and below are identical per taxon as an effect of the update phase (as discussed in Section 1). This is an important precondition met by Genestrip’s *hv_g* database to avoid false positives at read classification time. Nevertheless, data points from higher taxonomic ranks scatter because the complete viral database *cv_g* contains k -mers from many more nonhuman viruses, and these k -mers were pushed to higher ranks due to the lowest common ancestor update. Obviously, these additional k -mers are not required to classify reads accurately with respect to human viruses. The database *hv_g* contains only 3,865,987 k -mers with a disk size of approximately 53 MB, whereas *cv_g* has a disk size of 5.4 GB with almost 97 times as many k -mers.

KrakenUniq’s corresponding k -mer counts from Figure 5 (a) are not identical for the species rank and below. Although visually not well obvious in (a), this accounts for 158 scattering data points on the species rank and below. These differences offer

³The list of human viruses originates from [24]. Some human viruses known to be missing were manually added. All taxa were chosen on the species rank.

the potential for more error and in particular false positives at read classification time as will be exemplified next.

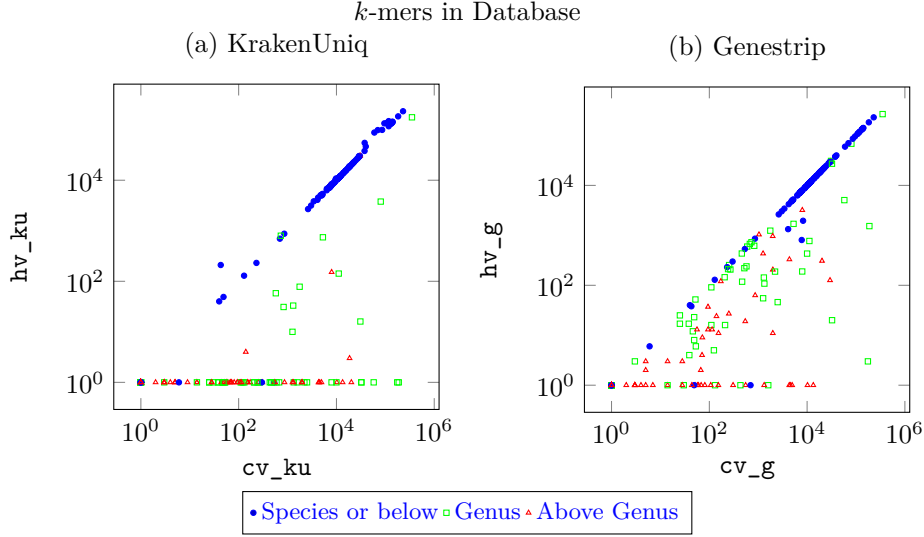


Fig. 5 Scatter plots of k -mers per taxon in a complete viral database versus a “human virus only” database both from the RefSeq created via (a) KrakenUniq and (b) Genestrip (based on a logarithmic scale with 1 added to each value to enable the representation of zero-counts).

Figure 6 (a,b) shows the scatter plots of k -mer counts and read counts from KrakenUniq on a logarithmic scale for the database cv_ku versus $h\nu_ku$ when applied to the human saliva fastq file #3 from Table 3. Figure 6 (c,d) shows the corresponding results with respect to the Genestrip databases cv_g versus $h\nu_g$: While the small database under KrakenUniq led to massive overcounts of k -mers for the rank species and below, this did not occur under Genestrip. Moreover, KrakenUniq’s overcounting of k -mers strongly influenced the counts of classified reads. Regarding Genestrip, the read counts on the species level and below are affected only by differing k -mer counts that occurred on the genus level and above. However, the impact remains moderate. The results for the other human saliva fastq files are similar (not shown).

In addition to the results from Figure 6, we measured the mean absolute percentage error of k -mer counts for all of the five human saliva fastq files from Table 3 between the complete viral database and the “human virus only” database in a variant where both databases belong to KrakenUniq and another variant where they both belong to Genestrip.

More precisely, we used the following error measure: Let S_d be the list of all taxa for the rank species or below in a database d . The mean absolute percentage error $err_{f,d}(e)$ for a database e with respect to fastq file f and the “ground truth” database

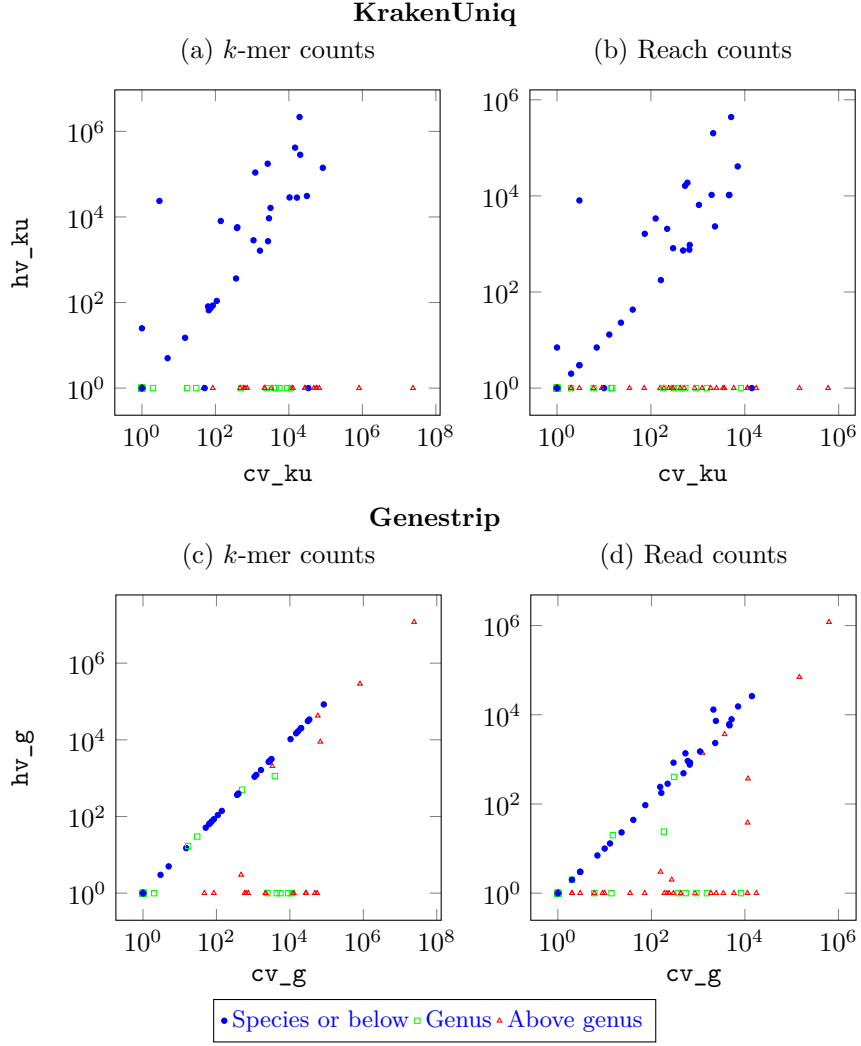


Fig. 6 Scatter plots of k -mer counts (a, c) and read counts (b, d) for the human saliva fastq file #3 from Table 3 analyzed with KrakenUniq (a, b) and Genestrip (c, d) using the complete viral databases versus the “human virus only” databases, as both shown in Figure 5 (based on a logarithmic scale with 1 added to each value to enable the representation of zero-counts).

d is as follows:⁴

$$err_{f,d}(e) := 100/|S_d \cap S_e| \cdot \sum_{t \in S_d \cap S_e} |k_{t,f,d} - k_{t,f,e}| / (k_{t,f,d} + 1) \quad (1)$$

Note that we limit the taxa to the rank species or below because, as discussed in the context of Figure 5 and 6, we cannot expect k -mer counts and read counts to be

⁴The term “ $k_{t,f,d} + 1$ ” in the formula includes a Laplace correction to avoid a potential division by zero.

close between d and a corresponding smaller database e for higher taxonomic ranks. Analogously, we also measured the error with regard to the counts of classified reads.

As shown in Table 4, the error rates for k -mer counts are zero under Genestrip, and for read counts, they are two orders of magnitude lower than those under KrakenUniq across all the examples. These findings solidify the usefulness of Genestrip’s analysis results in the context of smaller databases.

#	$err_{f,cv_ku}(hv_ku)$ for				$err_{f,cv_g}(hv_g)$ for			
	k -mers	(std.dev.)	Cl. reads	(std.dev.)	k -mers	(std.dev.)	Cl. reads	(std.dev.)
1	4,577.48	(58,525.07)	1,599.24	(19,829.44)	0.00	(0.00)	9.01	(43.41)
2	2,172.07	(27,925.48)	807.47	(9,280.12)	0.00	(0.00)	8.03	(41.81)
3	2,399.83	(30,825.50)	1,017.48	(11,979.38)	0.00	(0.00)	8.52	(41.37)
4	4,220.93	(56,213.73)	1,523.10	(19,100.24)	0.00	(0.00)	8.27	(44.44)
5	3,877.09	(51,694.66)	1,385.05	(17,330.62)	0.00	(0.00)	8.20	(42.46)

Table 4 Mean absolute percentage error of k -mer counts and read counts for five human saliva fastq files between the complete viral database and the “human virus only” database where both databases either belong to KrakenUniq or Genestrip, respectively. Standard deviations for the errors are included in parentheses.

3.3 Example: Tick-borne bacterial pathogens

This section presents the detection of tick-related bacterial pathogens as an example application of Genestrip. We compare the very large standard database **MicrobialDB** available for KrakenUniq⁵ against a relatively small Genestrip database **tb** with high genomic coverage regarding the organisms of interest. The results aim to show that a small but genomically deeper database such as **tb** can lead to higher recall than can KrakenUniq’s standard database while sustaining high precision.

An envisioned application is tick surveillance via metagenomics according to [11], where the authors applied Nanopore technology to check eight field-collected ticks but back then relied on Kraken 2 to identify corresponding organisms. Due to [26], [27] and [28] we also see a use case when screening for tick-borne pathogens in vertebrates potentially including humans.

3.3.1 The databases **MicrobialDB** and **tb**

MicrobialDB for KrakenUniq aims at the diagnosis of infectious diseases using metagenomics and contains 97.68 times as many k -mers as **tb**. It has a disk size of 574.31 GB excluding the taxonomy tree and additional mapping files. It covers a broad set of microbes including archaea, bacteria, viral genomes, the human genome and pathogenic eukaryotes with contaminants removed [17].

The database **tb** for Genestrip is based on the RefSeq Release 230 and covers all RefSeq genomes under the twelve bacterial genera *Bartonella*, *Rickettsia*, *Anaplasma*, *Ehrlichia*, *Borrelia*, *Borrelia*, *Chlamydia*, *Mycoplasma*, *Malacoplasma*, *Mycoplasma*, *Metamycoplasma* and *Francisella* as many included species are

⁵See “MicrobialDB” under [25].

pathogenic to humans and known or assumed to occur in ticks [29–33]. To avoid contaminants from human DNA and black-legged tick DNA, we added corresponding genomes to Genestrip’s update phase as explained at the end of Section 2.2. We also activated the genomic dust filter at high sensitivity: e.g., the k -mer ATACGGCGCGCGCGCGCATATATATTC would be dropped because its repetitiveness is just above the set threshold.

tb contains 487,834,649 k -mers and has a disk size of just 5.87 GB including the accompanied Bloom filter and the matching extract of the NCBI taxonomy tree. **tb** has 1.96 times as many k -mers stored under the selected genera and below as **MicrobialDB**. The macro average increase across the chosen genera is a factor of 1.83 with a maximum factor of 3.25 for *Bartonella* and a minimum factor of 1.29 for *Chlamydia*. The main reason behind this increase is the total growth of the RefSeq’s number of prokaryotic genomes by a factor of over 1.39 between the generation of **MicrobialDB** in mid 2023 and the generation of **tb** [7].

The generation of **tb** revealed that its included genomes from the RefSeq have a high overlap: On average each stored k -mer occurred 11.2 times in these genomes, as determined by the **tempindex** tool (see Section 2.2). Only 0.037 % of all k -mers considered for storing were dropped as “dust”. The update phase pushed 26.3 % of the stored k -mers to ranks above genus. 0.027 % of the stored k -mers had been pushed to the taxon of “cellular organisms” because tick DNA and human DNA were included on purpose during the update phase for the sake for contaminant detection.

3.3.2 Tick analysis

We applied **MicrobialDB** and **tb** to the eight fastq files from [11] mentioned above. The fastq files contain long reads from homogenized field-collected ticks sequenced via Nanopore technology.

As an example result, Figure 7 shows the five most frequent k -mer counts and reads counts for “Tick 6” accumulated up to the genus rank. In all the cases but “138” (*Borrelia*), the aggregated counts for the genera under **tb** are mildly to considerably higher than those achieved with **MicrobialDB**. Since we established the accuracy of Genestrip’s results in Section 3.1, this increase may be attributed with high confidence to the quality of the database **tb**, which is deeper than **MicrobialDB** for its included genera as described above. We observed similar improvements in terms of k -mers and reads with respect to *Ehrlichia*, *Rickettsia* and partly to *Anaplasma* in all of the eight ticks’ fastq files whenever the corresponding organisms showed presence via **MicrobialDB**.

Figure 8 details the results for Tick 6 with respect to “943” (*Ehrlichia*): Interestingly, Genestrip assigns most of the found k -mers right to the genus, whereas KrakenUniq produces a peak under the strain “1423892” (*Ehrlichia muris* AS145). The difference originates in the deeper database **tb**, where in comparison with **MicrobialDB**, more common k -mers were found from a broader range of genomic data under the genus *Ehrlichia* at **tb**’s generation time. In turn, Genestrip’s update phase pushed these k -mers to the genus rank as the lowest common ancestor. Indeed, the ratio of stored k -mers between **tb** and **MicrobialDB** is 4.54 for the taxon “943” but only 0.24 for “1423892”.

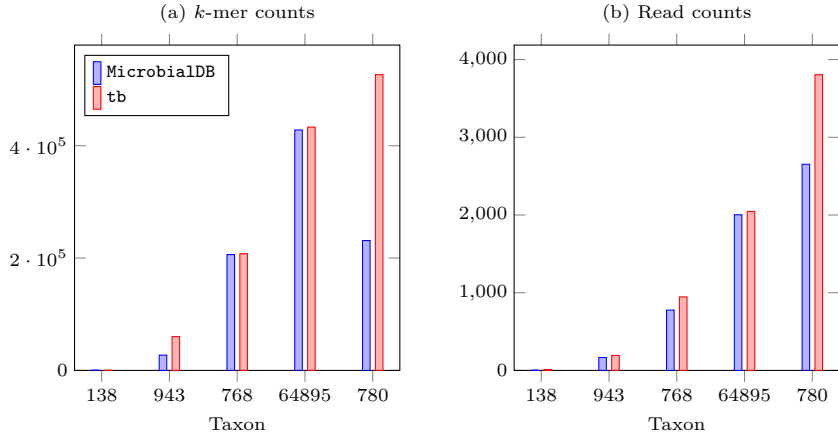


Fig. 7 Comparison of k -mer counts (a) and read counts (b) between the database **MicrobialDB** (based on KrakenUniq) and **tb** (based on Genestrip). The counts are aggregated up to rank genus for the long read fastq file of “Tick 6” from [11] and restricted to the genera included in **tb** with only the five most frequent genera in terms of k -mer counts shown.

Again, we observed similar results, particularly with respect to *Ehrlichia*, *Rickettsia* and partly to *Anaplasma*, in all of the eight ticks’ fastq files whenever corresponding organisms were present via **MicrobialDB**. These findings form a general trend when k -mer databases with a lowest common ancestor representation become deeper. The effect was first discovered and studied in [6]. Note that this is beneficial since such deeper k -mer databases tend to protect read classification from potentially falsely assigning a too low-ranked taxon.

3.4 Performance

As mentioned in Section 1, Genestrip’s focus is on generating and using potentially small but also deep k -mer databases for accurate metagenomic analysis of selected organisms of interest. This implies that users must be empowered to flexibly create new databases – ideally on their own devices. We therefore put focus on the performance of database generation as well as on the performance of read classification.

The following experiments were all performed on a modern PC with 32 GB of RAM, 8 CPU cores and a 2 TB solid state disk.⁶

3.5 Database generation

Table 5 lists six Genestrip databases used for assessing database generation performance. They refer to several categories and were designed with a potential use in medicine or agriculture in mind. Table 6 reports execution statistics for the generation of these databases from scratch via the goal **db** from Figure 1. The execution times

⁶We ensured that as few competing processes as possible were running during the experiments and OS disk swapping was disabled. Platform details: AMD Ryzen 7 2700X, Linux 6.12.37, OpenJDK 21.0.7 64-Bit Server VM.

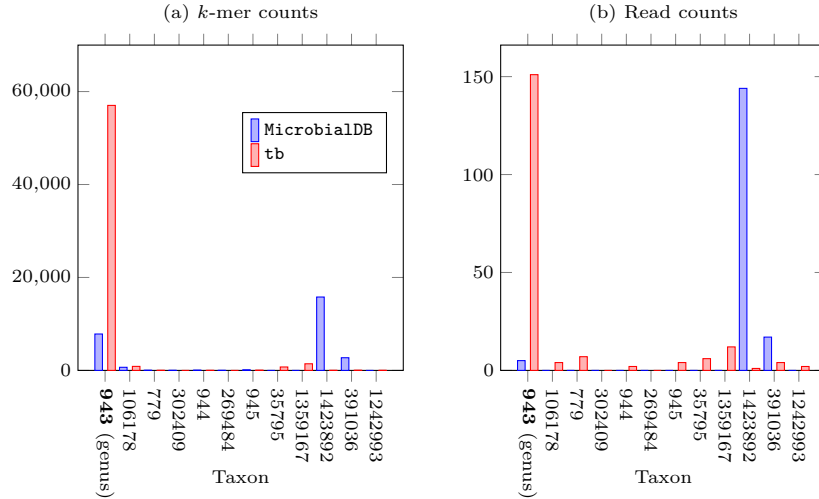


Fig. 8 Comparison of k -mer counts (a) and read counts (b) between the database **MicrobialDB** (based on KrakenUniq) and **tb** (based on Genestrip) for all taxons under the genus “943” (*Ehrlichia*) including the genus itself. The counts belong to the long read fastq file of “Tick 6” from [11] – the same file as in Figure 7.

and the maximum RAM remained moderate in all of the cases. (Times for downloading any of the necessary RefSeq or Genbank files from the NCBI are not included. Related files were downloaded upfront via the goals `refseqfna` and `fastasgenbankdl` from Figure 1.)

The different execution times in Table 6 depend on the size of the database itself but also on the RefSeq category, where “Bacteria” is the largest of the five ones used with genomes in over 2,800 zip files with a total disk size of over 550 GB (in RefSeq Release 230). The zip files must all be screened during each goal from Table 1 but the update phase, i.e., the goal `updatedb`, dominates the execution time.

DB name	Category	Description	Species and below
<code>cv_g</code>	Viral	See Section 3.1	25,739
<code>hv_g</code>	Viral	See Section 3.2	225
<code>tb</code>	Bacterial, Plasmid	See Section 3.3.1	1,119
<code>protozoa</code>	Protozoa	Comprehensive collection of protozoans from the RefSeq which are pathogenic to humans [34]	108
<code>vineyard</code>	Fungi	Fungal infections from the RefSeq and Genbank threatening grapevine plants [35]	11
<code>parasites</code>	Invertebrate	Comprehensive collection of parasitic invertebrates from the RefSeq which are pathogenic to humans [34]	77

Table 5 Background on the Genestrip databases used for assessing database generation performance.

DB name	Stored k -mers	Disk size (MB)	Wall time (min)	Max. RAM (MB)
cv_g	374,412,041	5,370	39	7,354
hv_g	3,865,987	53	5	1,014
tb	487,834,649	6,008	840	17,762
protozoa	1,279,065,780	15,915	88	22,148
vineyard	663,541,745	7,973	100	16,143
parasites	1,629,142,488	19,761	402	27,183

Table 6 Execution statistics for generating the six Genestrip databases from Table 5 using 10 threads and a maximum allowed JVM heap size of 24 GB.

3.6 Read classification

To assess Genestrip’s performance regarding k -mer counting and read classification, we compared its analysis runs with corresponding runs under KrakenUniq.

For the first comparison we applied the almost equally structured databases cv_ku for KrakenUniq and cv_g for Genestrip from Section 3.1 to the human saliva fastq files from Table 3. Scenario (a) of Table 7 summarizes the analysis performance across all five fastq files. Here, KrakenUniq offered about 33% better runtimes.

In scenario (b) of Table 7 we used the databases MicrobialDB vs. tb from Section 3.3.1 to analyze the eight tick-related fastq files from Section 3.3.2. Since MicrobialDB is so large, we had to make use of KrakenUniq’s database processing in chunks which broke down each run into 23 rounds. The chunk processing caused considerably longer run times. On the one hand, comparing the tools this way may be considered “unfair” because of unequally sized database sizes. On the other hand, there is no good alternative to employing a very large KrakenUniq database for the purpose (of detecting tick-related bacterial pathogens) because, as examined in Section 3.2, a small KrakenUniq database bares the high risk of many false positives.

In scenario (c) we had the fastq files from (b) jointly analyzed. In this case, Genestrip performed even better because the database had to be loaded only once, whereas in scenario (b), it got loaded separately for each fastq file. In contrast, KrakenUniq did not benefit from the joint analysis because its chunk processing had to be done all over again for each fastq file.

Regarding main memory, KrakenUniq’s and Genestrip’s analysis performance are dominated by the size of the database. In both cases, the required memory did not vary much across runs when the same database was used. Since MicrobialDB did not fit into memory, we set KrakenUniq’s preload size such that in total, it allocated most of the available memory. In case of the viral database, KrakenUniq still needs more RAM than Genestrip.

Overall, the results from both comparisons suggest that depending on the use case, Genestrip and its small databases offer competitive to highly favorable analysis performance.

4 Conclusions

We have presented Genestrip — a bioinformatics tool for (unique) k -mer counting, read classification and fastq file filtering. Its major improvement over existing systems

Scenario	Parameter	KrakenUniq	Genestrip
(a) <code>cv_ku</code> vs. <code>cv_g</code> from Section 3.1 on the five human saliva fastq files from Table 3	Avg. reads per fastq file	824,743,085	
	Avg. BPs per read	101	
	Avg. RAM (MB)	10,600	6,301
	Max. wall time (min)	28	41
	Avg. wall time (min)	23	34
	Avg. speed (reads / s)	597,639	404,285
(b) <code>MicrobialDB</code> vs. <code>tb</code> from Section 3.3.1 on the eight tick-related fastq files from Section 3.3.2	Min. reads per fastq file	644,767	
	Max. reads per fastq file	2,233,967	
	Avg. reads per fastq file	1,255,973	
	Avg. BPs per read	365	
	Avg. RAM (MB)	31,136	8,355
	Max. wall time (min)	16	1
	Avg. wall time (min)	14	1
	Avg. speed (reads / s)	1,495	27,010
(d) <code>MicrobialDB</code> vs. <code>tb</code> on the eight tick-related fastq files <i>jointly analyzed</i>	RAM (MB)	31,133	8,404
	Wall time (min)	114	2
	Speed (reads / s)	1,473	114,179

Table 7 Read analysis performance comparison between KrakenUniq and Genestrip using both tools with 10 threads. For `MicrobialDB` we set the parameter `--preload-size 24G` to limit KrakenUniq’s main memory according to [4]. (Otherwise, the analysis with KrakenUniq would have not been possible on the given PC.)

is its very low memory consumption at database generation time and at read analysis time while also maintaining very high accuracy. To achieve this, Genestrip facilitates and suggests the generation of small databases that cover the genomes of a collection of organisms of interest. Genestrip’s update phase ensures that during database generation, the most suitable lowest common ancestor taxon is computed for each stored k -mer by *also considering the genomes of organisms whose k -mers are not included in the database*. Normally, *Genestrip considers all genomes from a matching category of the RefSeq for this purpose*.

The results from Section 3.1 indicate that Genestrip attains the same type of correctness as KrakenUniq in terms of database structure, k -mer counting and read classification given that the databases of both systems are filled via all genomes belonging to an entire RefSeq category. But more importantly, Section 3.2 highlights that Genestrip sustains a high level accuracy when changing from a large, comprehensive database to a small database, although just a subset of the genomes out of a RefSeq category is included in that small database. In contrast, a small KrakenUniq database fails to achieve this level of accuracy because KrakenUniq’s lowest common ancestor computation does not ever consider genomes outside of the database to be generated.

Section 3.3 has exemplified the usefulness of Genestrip’s “small databases” in the context of tick surveillance where ticks were screened for a collection of well-known bacterial pathogens via metagenomics [11]. The resulting database `tb` has a disk size

of less than 6 GB and can be used for analysis on any modern PC. Nevertheless, Genestrip offers the exactness of KrakenUniq’s very large database **MicrobialDB** in terms of k -mer counting, unique k -mer analysis and read classification for the chosen genera.

Unlike Kraken 2, for example, Genestrip also eliminates the potential for false positive k -mer counts because it does not compress k -mers under loss of information. This renders our tool suitable in fields such as medical diagnostics where precision of k -mer counting and read classification is paramount — which was also a motive for evolving Kraken 1 into KrakenUniq [4] after the same research group had already authored Kraken 2 [2].

The deliberate focus on a particular set of genera or species allows for more genomes to be included for the organisms of interest while the resulting Genestrip database may still remain small. For example, the tick-related database **tb** from Section 3.3.1 contains 1.83 times more k -mers from the chosen genera than the latest publicly available large database **MicrobialDB** for KrakenUniq. As **tb** is deeper than **MicrobialDB** in this respect, it also tends to achieve better recall when analyzing tick related fastq files without loss of precision. We have presented and discussed the corresponding results as part of Section 3.3.2.

Regarding performance, we have examined *database generation as well as fastq file analysis*: Section 3.5 has shown that Genestrip empowers users to flexibly create new databases on regular PCs in just a few hours. As summarized in Tables 5 and 6, we created databases with different applications in mind where the generation of **tb** took the longest with still less than 14 hours while main memory consumption remained below 30 GB in all the cases.

Let us contrast Genestrip’s database generation performance with the corresponding resources required under KrakenUniq: As mentioned in Section 1.2, [6] stated that generating a full bacterial database for Kraken 1 and so equivalently for KrakenUniq from the RefSeq “required ca. 2.5 TB of RAM and ca. 11 days” on a suitable computer. However, since then, the RefSeq’s number of prokaryotic genomes alone has increased up by a factor of 4.5 to over 38 Mio. [7]. The additional growth of the RefSeq is also a likely reason why **MicrobialDB** has not been updated anymore since 2023 because ultralarge main memory and extremely high computational power would be required for this. Overall, this development suggests that k -mer databases with very broad and comprehensive coverage (such as by including *all* genomes from the RefSeq category “Bacteria”) have reached a “dead-end” — at least when a lossless storing of k -mers is desired.

In Section 3.6 we examined Genestrip’s analysis performance: When the database structure under KrakenUniq and Genestrip was similar then the performance was similar too with KrakenUniq offering about 33% better runtimes. However, when the use case for Genestrip’s small databases was met, then the database size played off with regard to the analysis time: E.g., when analyzing tick-borne bacterial pathogens via the database **tb**, Genestrip was more than ten times faster than KrakenUniq because on a regular PC, KrakenUniq’s large database **MicrobialDB** has to be processed in chunks over several iterations. In a similar scenario where related fastq files were jointly analyzed, Genestrip was over seventy times faster than KrakenUniq because

Genestrip spared reloading the database for each fastq file. Regarding main memory, Genestrip’s analysis performance is dominated by the size of the database itself. E.g, it remained below 9 GB of RAM under the database `tb` which has a disk size of about 6 GB.

In use cases where metagenomics is meant to be performed with very few restrictions on what organisms to look out for, then Genestrip may not be the right choice. Although it is certainly possible to create large databases under Genestrip, one would end up with similar challenges as with KrakenUniq (see above). If exactness of results is required in addition to broad coverage, then a two-stage analysis procedure might prove useful: In stage one, a tool such as Kraken 2 with a compressed database for broad coverage may be applied to identify candidate organisms of interest. In stage two, Genestrip might be used to confirm and “drill down” the initial findings by potentially first generating and then using a dedicated database.

5 Availability and requirements

Project name: Genestrip

Project home page: <https://github.com/pfeiferd/genestrip>

Operating system(s): Platform independent

Programming language: Java

Execution requirements: JRE 11 or higher

Build requirements: JDK 11 or higher and Maven 2 or 3

License: Apache 2.0 subject to the Commons Clause License Condition v1.0

Any restrictions to use by non-academics: License needed for commercial use

6 List of abbreviations

t	Placeholder for taxon
f	Placeholder for fastq file
d	Placeholder for k -mer database
$k_{t,f,d}$	Number of k -mers found for taxon t in fastq file f via database d
$u_{t,f,d}$	Number of unique k -mers found for taxon t in fastq file f via database d
$u_{t,d}$	Number of k -mers stored under taxon t in database d
cv_ku	Complete viral database for KrakenUniq
cv_g	Complete viral database for Genestrip
BPs	Base pairs
hv_ku	Human virus database for KrakenUniq
hv_g	Human virus database for Genestrip
S_d	Set of taxons in database d for the rank species or below
$err_{f,d}(e)$	Mean absolute percentage error for the analysis of fastq file f via database e where corresponding results based on database d are the ground truth
MicrobialDB	Large, comprehensive microbial database for KrakenUniq
tb	Tick-borne bacterial pathogens database for Genestrip
DB	Database

7 Declarations

7.1 Ethics approval and consent to participate

Not applicable

7.2 Consent for publication

Not applicable

7.3 Availability of data and materials

The exact code base for this publication is available under <https://github.com/pfeiferd/genestrip/releases/tag/v2.2> and <https://doi.org/10.5281/zenodo.16758120>.

The code base used to generate all of the results from this publication is available under:

<https://github.com/pfeiferd/genestrip-db-exp/tag/v1.6> and <https://doi.org/10.5281/zenodo.16812481>.

Following [36], the human saliva fastq files were acquired from the Sequence Read Archive (SRA) under the entries: ERX1462737, ERX1462740, SRX2830683, SRX2830684, and SRX2830689. Following [11], the tick-related fastq files were acquired from the Sequence Read Archive (SRA) under the entries SRX13458610, SRX13458622, SRX13458624, SRX13458626, SRX13458627, SRX13458628, SRX13458611 and SRX13458612.

7.4 Competing interests

The authors declare that they have no competing interests.

7.5 Funding

The work behind this publication was not funded.

7.6 Authors' contributions

DP developed the software, designed the evaluation and wrote the manuscript. CR conducted the performance experiments and compiled the respective results.

7.7 Acknowledgements

We thank Dr. Richard Zowalla, Simon Schweizer and Jochen Schäfer for providing server-based environments enabling the development and testing of Genestrip. We thank Dr. Robert Friedrich and Lennart Elze for their feedback when initially applying the software. Many thanks also to Dr. Markus Graf for proofreading the manuscript.

References

- [1] Wood DE, Salzberg SL. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*. 2014;15(3):R46. <https://doi.org/10.1186/gb-2014-15-3-r46>.
- [2] Wood DE, Lu J, Langmead B. Improved metagenomic analysis with Kraken 2. *Genome Biology*. 2019;20(1):257. <https://doi.org/10.1186/s13059-019-1891-0>.
- [3] Breitwieser FP, Baker DN, Salzberg SL. KrakenUniq: confident and fast metagenomics classification using unique k-mer counts. *Genome Biology*. 2018;19(1):198. <https://doi.org/10.1186/s13059-018-1568-0>.
- [4] Pockrandt C, Zimin AV, Salzberg SL. Metagenomic classification with KrakenUniq on low-memory computers. *J Open Source Softw*. 2022;7(80). <https://doi.org/10.21105/joss.04908>.
- [5] O’Leary NA, Wright MW, Brister JR, Ciufu S, Haddad D, McVeigh R, et al. Reference sequence (RefSeq) database at NCBI: current status, taxonomic expansion, and functional annotation. *Nucleic Acids Res*. 2016 Jan;44(D1):D733–45. <https://doi.org/10.1093/nar/gkv1189>.
- [6] Nasko DJ, Koren S, Phillippy AM, Treangen TJ. RefSeq database growth influences the accuracy of k-mer-based lowest common ancestor species identification. *Genome Biology*. 2018;19(1):165. <https://doi.org/10.1186/s13059-018-1554-6>.
- [7] Goldfarb T, Kodali V, Pujar S, Brover V, Robbertse B, Farrell C, et al. NCBI RefSeq: reference sequence standards through 25 years of curation and annotation. *Nucleic Acids Research*. 2024 11;53(D1):D243–D257. <https://doi.org/10.1093/nar/gkae1038>. <https://academic.oup.com/nar/article-pdf/53/D1/D243/60586509/gkae1038.pdf>.
- [8] Sayers EW, Cavanaugh M, Clark K, Ostell J, Pruitt KD, Karsch-Mizrachi I. GenBank. *Nucleic Acids Research*. 2019 10;48(D1):D84–D86. <https://doi.org/10.1093/nar/gkz956>. <https://academic.oup.com/nar/article-pdf/48/D1/D84/31697602/gkz956.pdf>.
- [9] de la Cuesta-Zuluaga J, Ley RE, Youngblut ND. Struo: a pipeline for building custom databases for common metagenome profilers. *Bioinformatics*. 2020 Apr;36(7):2314–2315. <https://doi.org/10.1093/bioinformatics/btz899>.
- [10] Youngblut ND, Ley RE. Struo2: efficient metagenome profiling database construction for ever-expanding microbial genome datasets. *PeerJ*. 2021;9:e12198. <https://doi.org/10.7717/peerj.12198>.

- [11] Kipp EJ, Lindsey LL, Khoo B, Faulk C, Oliver JD, Larsen PA. Metagenomic surveillance for bacterial tick-borne pathogens using nanopore adaptive sampling. *Scientific Reports*. 2023;13(1):10991. <https://doi.org/10.1038/s41598-023-37134-9>.
- [12] Feldman SI. Make – a program for maintaining computer programs. *Software: Practice and experience*. 1979;9(4):255–265.
- [13] Sayers EW, Beck J, Bolton EE, Brister JR, Chan J, Connor R, et al. Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Res*. 2025 Jan;53(D1):D20–D29. <https://doi.org/10.1093/nar/gkae979>.
- [14] Bloom BH. Space/time trade-offs in hash coding with allowable errors. *Commun ACM*. 1970 Jul;13(7):422–426. <https://doi.org/10.1145/362686.362692>.
- [15] Schoch CL, Ciufo S, Domrachev M, Hottel CL, Kannan S, Khovanskaya R, et al. NCBI Taxonomy: a comprehensive update on curation, resources and tools. *Database (Oxford)*. 2020 Jan;2020. <https://doi.org/10.1093/database/baaa062>.
- [16] Breitwieser FP, Pertea M, Zimin AV, Salzberg SL. Human contamination in bacterial genomes has created thousands of spurious proteins. *Genome Res*. 2019 Jun;29(6):954–960. <https://doi.org/10.1101/gr.245373.118>.
- [17] Lu J, Salzberg SL. Removing contaminants from databases of draft genomes. *PLoS Comput Biol*. 2018 Jun;14(6):e1006277. <https://doi.org/10.1371/journal.pcbi.1006277>.
- [18] Morgulis A, Gertz EM, Schäffer AA, Agarwala R. A Fast and Symmetric DUST Implementation to Mask Low-Complexity DNA Sequences. *Journal of Computational Biology*. 2006;13(5):1028–1040. PMID: 16796549. <https://doi.org/10.1089/cmb.2006.13.1028>. <https://doi.org/10.1089/cmb.2006.13.1028>.
- [19] Chu J, Sadeghi S, Raymond A, Jackman SD, Nip KM, Mar R, et al. BioBloom tools: fast, accurate and memory-efficient host species sequence screening using bloom filters. *Bioinformatics*. 2014 08;30(23):3402–3404. <https://doi.org/10.1093/bioinformatics/btu558>. https://academic.oup.com/bioinformatics/article-pdf/30/23/3402/48931434/bioinformatics_30_23_3402.pdf.
- [20] Delahaye C, Nicolas J. Sequencing DNA with nanopores: Troubles and biases. *PLoS One*. 2021;16(10):e0257521. <https://doi.org/10.1371/journal.pone.0257521>.
- [21] Liu-Wei W, van der Toorn W, Bohn P, Hölzer M, Smyth RP, von Kleist M. Sequencing accuracy and systematic errors of nanopore direct RNA sequencing. *BMC Genomics*. 2024;25(1):528. <https://doi.org/10.1186/s12864-024-10440-w>.
- [22] Lu J, Rincon N, Wood DE, Breitwieser FP, Pockrandt C, Langmead B, et al. Metagenome analysis using the Kraken software suite. *Nature Protocols*.

2022;17(12):2815–2839. <https://doi.org/10.1038/s41596-022-00738-y>.

- [23] Wu LY, Wijesekara Y, Piedade GJ, Pappas N, Brussaard CPD, Dutilh BE. Benchmarking bioinformatic virus identification tools using real-world metagenomic data across biomes. *Genome Biology*. 2024;25(1):97. <https://doi.org/10.1186/s13059-024-03236-4>.
- [24] Hulo C, de Castro E, Masson P, Bougueleret L, Bairoch A, Xenarios I, et al. ViralZone: a knowledge resource to understand virus diversity. *Nucleic Acids Res*. 2011 Jan;39(Database issue):D576–82. <https://doi.org/10.1093/nar/gkq901>.
- [25] Langmead B.: Kraken 2, KrakenUniq and Bracken indexes. <https://benlangmead.github.io/aws-indexes/k2>.
- [26] Huggins LG, Namgyel U, Wangchuk P, Atapattu U, Traub R, Colella V. Metabarcoding using nanopore sequencing enables identification of diverse and zoonotic vector-borne pathogens from neglected regions: A case study investigating dogs from Bhutan. *One Health*. 2024;19:100839. <https://doi.org/https://doi.org/10.1016/j.onehlt.2024.100839>.
- [27] Miao Y, Guo W, Zhang W, Chen Z, Mian D, Li R, et al. Detection of *Bartonella* spp. in farmed deer (*Artiodactyla*: *Cervidae*) using multiplex assays in the Qinghai-Tibet Plateau, China. *Microbiol Spectr*. 2024 Jul;12(7):e0412023. <https://doi.org/10.1128/spectrum.04120-23>.
- [28] Huggins LG, Colella V, Young ND, Traub RJ. Metabarcoding using nanopore long-read sequencing for the unbiased characterization of apicomplexan haemoparasites. *Mol Ecol Resour*. 2024 Feb;24(2):e13878. <https://doi.org/10.1111/1755-0998.13878>.
- [29] Eisen RJ, Eisen L. The Blacklegged Tick, *Ixodes scapularis*: An Increasing Public Health Concern [doi: 10.1016/j.pt.2017.12.006]. *Trends in Parasitology*. 2018 2025/06/22;34(4):295–309. <https://doi.org/10.1016/j.pt.2017.12.006>.
- [30] Johnson TL, Graham CB, Maes SE, Hojgaard A, Fleshman A, Boegler KA, et al. Prevalence and distribution of seven human pathogens in host-seeking *Ixodes scapularis* (*Acari*: *Ixodidae*) nymphs in Minnesota, USA. *Ticks and Tick-borne Diseases*. 2018;9(6):1499–1507. <https://doi.org/https://doi.org/10.1016/j.ttbdis.2018.07.009>.
- [31] Janecek E, Mietze A, Goethe R, Schnieder T, Strube C. *Bartonella* spp. infection rate and *B. grahamii* in ticks. *Emerg Infect Dis*. 2012 Oct;18(10):1689–1690. <https://doi.org/10.3201/eid1810.120390>.
- [32] Thomas RE, Karstens RH, Schwan TG. Experimental infection of *Ornithodoros* spp. ticks (*Acari*: *Argasidae*) with *Yersinia pestis*. *J Med Entomol*. 1990 Jul;27(4):720–723. <https://doi.org/10.1093/jmedent/27.4.720>.

- [33] Shi H, Li G, Li D, Zhai H, Ji S, Hu Y, et al. Molecular investigation reveals three hemotropic mycoplasmas in cats and three tick species in China. *Frontiers in Veterinary Science*. 2025;Volume 12 - 2025. <https://doi.org/10.3389/fvets.2025.1522904>.
- [34] Mehlhorn H. *Die Parasiten des Menschen: Erkrankungen erkennen, bekämpfen und vorbeugen*. SpringerLink: Bücher. Heidelberg: Springer Spektrum; 2023.
- [35] Hillebrand W, Lorenz D, Louis F. *Rebschutz-Taschenbuch*. Heidelberg: Fachverlag Fraund; 1998.
- [36] Samson CA, Whitford W, Snell RG, Jacobsen JC, Lehnert K. Contaminating DNA in human saliva alters the detection of variants from whole genome sequencing. *Sci Rep*. 2020 Nov;10(1):19255. <https://doi.org/10.1038/s41598-020-76022-4>.